

Respuestas - Conocimientos SQL

Evaluación Técnica de Programación · Grupo Castores

Candidato	Pablo Arámbula Ortiz
Sección	1. Conocimientos SQL (1.1 a 1.7)

Objetivo. Responder los conceptos fundamentales de SQL solicitados y presentar las consultas correspondientes a las tablas productos y ventas del ejercicio.

1.1 Describe el funcionamiento general de la sentencia JOIN.

La sentencia JOIN permite combinar filas de dos o más tablas relacionadas mediante una condición de unión. Normalmente, la relación se establece entre una clave primaria y una clave foránea. El resultado contiene columnas de las tablas involucradas y las filas que cumplen las reglas del tipo de JOIN utilizado.

Ejemplo: relacionar productos con sus ventas mediante idProducto.

```
SELECT p.nombre, v.cantidad
FROM productos AS p
INNER JOIN ventas AS v
    ON p.idProducto = v.idProducto;
```

1.2 ¿Cuáles son los tipos de JOIN y cuál es el funcionamiento de los mismos?

Los tipos principales son:

INNER JOIN. Devuelve únicamente las filas que tienen coincidencia en ambas tablas.

```
SELECT *
FROM productos AS p
INNER JOIN ventas AS v
    ON p.idProducto = v.idProducto;
```

LEFT JOIN. Devuelve todas las filas de la tabla izquierda y las coincidencias de la tabla derecha. Cuando no existe coincidencia, las columnas de la tabla derecha se muestran como NULL.

```
SELECT *
FROM productos AS p
LEFT JOIN ventas AS v
    ON p.idProducto = v.idProducto;
```

RIGHT JOIN. Devuelve todas las filas de la tabla derecha y las coincidencias de la tabla izquierda. Cuando no existe coincidencia, las columnas de la tabla izquierda se muestran como NULL.

```
SELECT *
FROM productos AS p
RIGHT JOIN ventas AS v
ON p.idProducto = v.idProducto;
```

FULL OUTER JOIN. Devuelve todas las filas de ambas tablas, coincidan o no. SQL Server lo soporta de forma nativa. En MySQL puede simularse combinando LEFT JOIN y RIGHT JOIN mediante UNION.

```
SELECT *
FROM productos AS p
FULL OUTER JOIN ventas AS v
ON p.idProducto = v.idProducto;
```

1.3 ¿Cuál es el funcionamiento general de los TRIGGER y qué propósito tienen?

Un TRIGGER es un objeto de base de datos que se ejecuta automáticamente cuando ocurre un evento específico sobre una tabla o vista, por ejemplo INSERT, UPDATE o DELETE. Su propósito es automatizar acciones vinculadas a cambios en los datos, como auditorías, validaciones, mantenimiento de históricos o aplicación de reglas de integridad.

Ejemplo conceptual: registrar automáticamente en una tabla de auditoría cuando un producto es actualizado.

```
CREATE TRIGGER trg_producto_actualizado
AFTER UPDATE ON productos
FOR EACH ROW
BEGIN
    INSERT INTO auditoria(descripcion, fecha)
    VALUES ('Producto actualizado', CURRENT_TIMESTAMP);
END;
```

Nota: la sintaxis exacta de los TRIGGER puede variar entre MySQL y SQL Server.

1.4 ¿Qué es y para qué sirve un STORED PROCEDURE?

Un STORED PROCEDURE (procedimiento almacenado) es un conjunto de instrucciones SQL guardadas y ejecutadas dentro del servidor de base de datos. Puede recibir parámetros, realizar consultas, validaciones, actualizaciones y transacciones, y devolver resultados.

Sirve principalmente para centralizar lógica de datos, reutilizar operaciones, controlar transacciones, mejorar la seguridad al limitar el acceso directo a tablas y reducir duplicación de código.

```
CREATE PROCEDURE aumentar_inventario(
    IN p_idProducto INT,
    IN p_cantidad INT
)
BEGIN
    UPDATE productos
    SET cantidad = cantidad + p_cantidad
    WHERE idProducto = p_idProducto;
END;
```

Consultas SQL del ejercicio

Las consultas siguientes consideran las tablas proporcionadas en la evaluación: productos(idProducto, nombre, precio) y ventas(idVenta, idProducto, cantidad).

1.5 Traer todos los productos que tengan una venta.

Se utiliza INNER JOIN porque solamente interesan productos que tengan al menos una coincidencia en ventas. DISTINCT evita repetir un producto cuando aparece en más de una venta.

```
SELECT DISTINCT
  p.idProducto,
  p.nombre,
  p.precio
FROM productos AS p
INNER JOIN ventas AS v
  ON p.idProducto = v.idProducto;
```

1.6 Traer todos los productos que tengan ventas y la cantidad total de productos vendidos.

Se agrupan las ventas por producto y se utiliza SUM sobre la columna cantidad para obtener el total de unidades vendidas de cada producto.

```
SELECT
  p.idProducto,
  p.nombre,
  p.precio,
  SUM(v.cantidad) AS cantidadTotalVendida
FROM productos AS p
INNER JOIN ventas AS v
  ON p.idProducto = v.idProducto
GROUP BY
  p.idProducto,
  p.nombre,
  p.precio;
```

1.7 Traer todos los productos, tengan ventas o no, y la suma total (\$) vendida por producto.

Se utiliza LEFT JOIN para conservar todos los productos. El importe vendido por fila se obtiene multiplicando precio por cantidad; después se suma por producto. COALESCE convierte el valor NULL de los productos sin ventas en 0.

```
SELECT
  p.idProducto,
  p.nombre,
  p.precio,
  COALESCE(SUM(p.precio * v.cantidad), 0) AS totalVendido
FROM productos AS p
LEFT JOIN ventas AS v
  ON p.idProducto = v.idProducto
GROUP BY
  p.idProducto,
  p.nombre,
  p.precio;
```